ByteMart Digital

THE COMPLETE GUIDE

AI with Spring Boot

Building AI Applications in Java with Spring AI

A complete, example-driven guide to building AI-powered applications in Java with Spring AI — from ChatClient and prompts to embeddings, RAG, tool calling, agents, MCP, and production.

24	110+	11	6
Chapters	Code Examples	Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

AI with Spring Boot & Java: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided "as is".

Trademarks
Spring, Spring AI, OpenAI, Anthropic, Ollama, and other product names referenced are the property of their respective owners and are used for identification purposes only. This guide is an independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. AI for Java Developers 4
2. Introducing Spring AI 6
3. Getting Started 8
4. Models & Providers 11
5. The ChatClient API 14
6. Prompts & Messages 17
7. Prompt Templates 20
8. Structured Output 23
9. Streaming Responses 26
10. Embeddings 28
11. Vector Stores 31
12. Retrieval-Augmented Generation (RAG) 34
13. The Document ETL Pipeline 37
14. Advanced RAG 40
15. Function / Tool Calling 42
16. Chat Memory & Conversations 45
17. The Advisors API 48
18. Model Context Protocol (MCP) 51
19. Building Agents 54
20. Multimodality & Image Models 56
21. Audio: Transcription & Speech 58
22. Working with Local Models 60
23. Observability, Evaluation & Testing 62
24. Production Concerns & Best Practices 64

MODULE 1 · FOUNDATIONS

CHAPTER 01

AI for Java Developers

What large language models offer a backend engineer, and where they fit in a Spring application.

You do not need a data-science background to build AI features. Modern **large language models** (LLMs) are consumed like any other remote service — you send a prompt over an API and get text back — which makes them a natural fit for a Spring Boot backend.

> **Large Language Model**
> A model trained on vast amounts of text that predicts and generates language. Exposed through an API, it can answer questions, summarise, classify, extract data, write code, and call tools — all from a natural-language prompt.

The mental shift

A traditional API returns deterministic, structured data. An LLM returns **generated** text that varies between calls and can be wrong. Building with AI means embracing that probabilistic nature — grounding the model in your data, validating its output, and designing for the occasional mistake.

What LLMs let you build

- **Chat and assistants** — answer questions over your documents or product.
- **Extraction** — turn unstructured text into typed Java objects.
- **Classification and routing** — categorise tickets, emails, content.
- **Summarisation and generation** — condense or draft text.
- **Agents** — models that call your code to get things done.

Where it fits in Spring

AI features live in the same service layer as the rest of your logic. A controller receives a request, a service calls the model (and your database, and other services), and you return a response — the model is just one more dependency, albeit an unusual one.

> **LLMs are a backend dependency**
> For a Java developer, an LLM is a remote service that turns prompts into generated text. The skill is not training models but integrating them well — grounding, structuring, and validating their output inside a Spring application.

© 2026 ByteMart Digital

MODULE 1 · FOUNDATIONS

CHAPTER 02

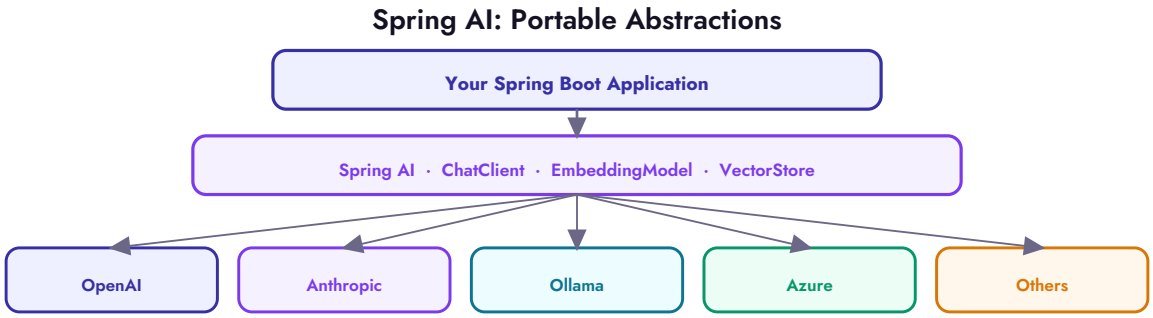
Introducing Spring AI

The framework's portable abstractions that let you build AI features without vendor lock-in.

Spring AI is the Spring project for building AI-powered applications. It brings the familiar Spring philosophy — portable abstractions, auto-configuration, and starters — to models, embeddings, vector stores, and more.

Portable abstractions

Spring AI defines provider-neutral interfaces such as `ChatModel`, `EmbeddingModel`, `ImageModel`, and `VectorStore`. You write against these, and swap the underlying provider — OpenAI, Anthropic, Ollama, Azure — through configuration, not code changes.



Write once against Spring AI's API; swap model providers via configuration.

Your app targets Spring AI's abstractions; the provider is chosen by configuration and starters.

What Spring AI gives you

- A fluent `ChatClient` for prompting models.
- Structured output — map replies straight to Java objects.
- Embeddings and vector stores for semantic search and RAG.
- Tool (function) calling so models can invoke your code.
- Chat memory, advisors, observability, and evaluation.

Built the Spring way

Everything is a Spring bean, auto-configured from properties, and injected where you need it. If you know Spring Boot, Spring AI will feel immediately familiar — add a starter, set an API key, and inject a client.

> **Spring conventions for AI**
> Spring AI wraps AI providers behind portable abstractions and auto-configuration. You build features against ChatModel, EmbeddingModel, and VectorStore, and switch providers without rewriting code.

MODULE 1 · FOUNDATIONS

CHAPTER 03

Getting Started

Adding a starter, configuring a provider, and making your first model call.

Getting a Spring Boot app talking to a model takes three steps: add a starter, configure a provider, and inject a client. Let us walk through the smallest possible example.

Add a model starter

Each provider has a Spring Boot starter that auto-configures the model beans. For OpenAI:

```xml
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-openai</artifactId>
</dependency>
```

Configure the provider

Provide the API key (and optional model settings) in `application.properties`. Keep secrets in environment variables, not source control.

```properties
spring.ai.openai.api-key=${OPENAI_API_KEY}
spring.ai.openai.chat.options.model=gpt-4o
spring.ai.openai.chat.options.temperature=0.7
```

Your first call

Inject a `ChatClient.Builder`, build a client, and prompt the model. Spring AI wires up the rest.

```java
@RestController
class ChatController {
    private final ChatClient chat;

    ChatController(ChatClient.Builder builder) {
        this.chat = builder.build();
    }

    @GetMapping("/ask")
    String ask(@RequestParam String q) {
        return chat.prompt().user(q).call().content();
    }
}
```

© 2026 ByteMart Digital

TIP

For local, free experimentation without an API key, use the Ollama starter to run open models like Llama on your own machine — the same code, a different provider.

Starter, key, client

Add a model starter, set the API key in properties, and inject ChatClient.Builder. Three steps and your Spring Boot app is calling an LLM.

© 2026 ByteMart Digital

MODULE 1 · FOUNDATIONS

CHAPTER 04

Models & Providers

The model types Spring AI abstracts, and how provider portability actually works.

Spring AI is not tied to one vendor. It abstracts several kinds of model behind interfaces, each with implementations for the major providers.

The model interfaces

Interface	Purpose
ChatModel	text generation / conversation
EmbeddingModel	text -> vector embeddings
ImageModel	image generation
Transcription / Speech	audio to text and back

Providers

Each interface has provider implementations — OpenAI, Anthropic, Azure OpenAI, Google Vertex, Amazon Bedrock, Ollama, Mistral, and more. You choose one by adding its starter and setting its properties; the rest of your code is unchanged.

How portability helps

- **Avoid lock-in** — switch providers if pricing or quality changes.
- **Mix providers** — a cheap local model for simple tasks, a strong one for hard tasks.
- **Test cheaply** — develop against a local Ollama model, deploy against a hosted one.

Setting options per call

Defaults come from properties, but you can override options (model, temperature, max tokens) per request through the `ChatClient` or a `ChatOptions` object — useful when different endpoints need different behaviour.

```java
chat.prompt()
    .user(q)
    .options(ChatOptions.builder().temperature(0.2).build())
    .call().content();
```

One API, many providers

Spring AI abstracts chat, embedding, image, and audio models behind interfaces with implementations for every major provider. Configuration — not code — selects the provider.

© 2026 ByteMart Digital

MODULE 2 · CHAT & PROMPTS

CHAPTER 05

The ChatClient API

The fluent client for prompting models, with synchronous and streaming calls.

The `ChatClient` is your main entry point for talking to a chat model. Its fluent API reads naturally: build a prompt, call the model, and extract the reply.

The basic shape

```java
String answer = chatClient.prompt()
    .system("You are a concise assistant.")
    .user("Explain dependency injection in one sentence.")
    .call()
    .content();
```

The ChatClient Request Flow



call() -> a single response | stream() -> a Flux of tokens

The fluent ChatClient builds the prompt, runs advisors, calls the model, maps the reply.

The ChatClient builds the prompt, runs advisors, calls the model, and maps the reply.

call() vs stream()

`call()` returns the whole response once the model finishes. `stream()` returns a reactive `Flux<String>` that emits tokens as they are generated — ideal for a responsive, typewriter-style UI.

```java
Flux<String> tokens = chatClient.prompt()
    .user(q)
    .stream()
    .content();
```

Getting more than text

Beyond `content()`, you can retrieve the full `ChatResponse` (with token usage and metadata) via `chatResponse()`, or bind the reply to a typed object via `entity(...)` — covered in the structured-output chapter.

Default configuration

Configure a client once with default system prompts, advisors, and options using `ChatClient.Builder`, then reuse it. Per-request settings override the defaults.

Fluent prompting

ChatClient.prompt()…call().content() is the core pattern. Use call() for a complete response and stream() for token-by-token output; configure sensible defaults once on the builder.

MODULE 2 · CHAT & PROMPTS

CHAPTER 06

Prompts & Messages

06

System, user, and assistant messages, and how they shape the model's behaviour.

A prompt is more than a question — it is a structured conversation made of typed **messages**. Understanding the message roles is the key to controlling a model's behaviour.

The message roles

Message	Role
SystemMessage	instructions / persona (how to behave)
UserMessage	the user's input
AssistantMessage	the model's prior replies

The system message

The `SystemMessage` sets the ground rules — the assistant's persona, tone, constraints, and output format. It is the single most effective lever for steering behaviour, and it applies to the whole conversation.

java

```
chatClient.prompt()
    .system("""
        You are a Java expert. Answer concisely.
        If unsure, say so rather than guessing.
        """)
    .user(question)
    .call().content();
```

Building a Prompt directly

For full control you can construct a `Prompt` from a list of messages and pass it to the model — useful when replaying a conversation history.

java

```
Prompt prompt = new Prompt(List.of(
    new SystemMessage("You are helpful."),
    new UserMessage("Hello")));
ChatResponse resp = chatModel.call(prompt);
```

WATCH OUT

The model has no memory between calls — each request is independent. To continue a conversation, you must send the prior messages again (see the chat-memory chapter).

Roles shape behaviour

A prompt is a sequence of system, user, and assistant messages. The system message sets persona and rules; the model treats each call as independent unless you resend history.

© 2026 ByteMart Digital

MODULE 2 · CHAT & PROMPTS

CHAPTER 07

Prompt Templates

Parameterising prompts with variables for reusable, maintainable prompting.

Hard-coding prompts with string concatenation gets messy fast. **Prompt templates** let you define a prompt with placeholders and fill them with variables — clean, reusable, and testable.

A basic template

```java
PromptTemplate template = new PromptTemplate("""
    Summarise the following {style} for a {audience}:

    {text}
    """);

Prompt prompt = template.create(Map.of(
    "style", "article",
    "audience", "beginner",
    "text", article));
```

Templating in the ChatClient

The `ChatClient` supports templating inline — pass the template text and the variables, and Spring AI substitutes them.

```java
chatClient.prompt()
    .user(u -> u.text("Translate '{text}' to {lang}")
                .param("text", text)
                .param("lang", "French"))
    .call().content();
```

Externalising prompts

Store prompt templates as classpath resources so they can be edited, reviewed, and version-controlled separately from code — treating prompts as first-class, evolving assets.

```java
@Value("classpath:/prompts/summarise.st")
private Resource summarisePrompt;
// new PromptTemplate(summarisePrompt) ...
```

TIP

Keep prompts in files, not code. Prompt engineering is iterative — externalising them lets non-developers tune wording without a rebuild.

Prompts with variables

Prompt templates parameterise prompts with placeholders, keeping them reusable and maintainable. Externalise templates as resources so prompts can evolve independently of code.

© 2026 ByteMart Digital

MODULE 2 · CHAT & PROMPTS

CHAPTER 08

Structured Output

Binding a model's free-text reply directly to typed Java objects.

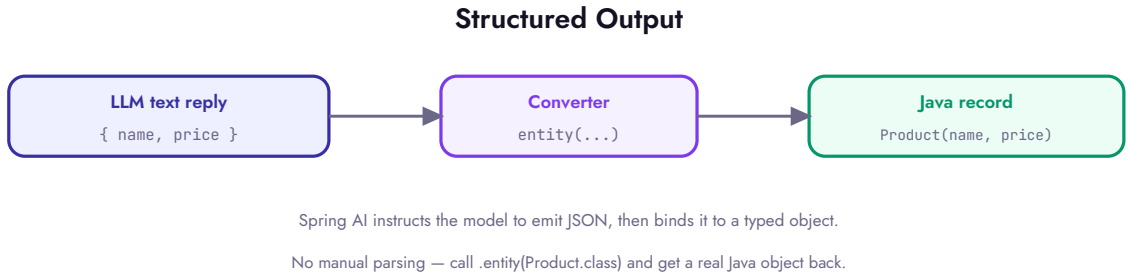
An LLM returns text, but your application wants objects. Spring AI's **structured output** bridges the gap — it instructs the model to produce parseable output and binds it to a Java type automatically.

Binding to a record

Call `.entity(...)` with a target type. Spring AI adds format instructions to the prompt and converts the reply into that type.

```java
record MovieReview(String title, int rating, String summary) {}

MovieReview review = chatClient.prompt()
    .user("Review the film Inception in JSON.")
    .call()
    .entity(MovieReview.class);
```



Spring AI instructs the model to emit JSON, then binds it to a typed Java object.

Collections and maps

Structured output works for lists and maps too, using generic type hints so the converter knows the element type.

```java
List<Movie> movies = chatClient.prompt()
    .user("List 5 sci-fi films.")
    .call()
    .entity(new ParameterizedTypeReference<List<Movie>>() {});
```

How it works

Under the hood an `OutputConverter` appends JSON-schema-style format instructions to your prompt and parses the model's reply. It is convenient but not infallible — validate critical fields, since the model can still

stray.

WATCH OUT

Structured output improves reliability but does not guarantee valid output every time. Validate parsed objects and handle conversion failures gracefully.

Text in, objects out

Call .entity(Type.class) to bind a model reply to a Java record, list, or map. Spring AI injects format instructions and parses the response — validate the result for critical use.

© 2026 ByteMart Digital

MODULE 2 · CHAT & PROMPTS

CHAPTER 09

Streaming Responses

Emitting model output token by token for a responsive user experience.

Waiting several seconds for a full LLM response feels sluggish. **Streaming** emits the reply as it is generated, so the user sees text appear immediately — the familiar typewriter effect.

Streaming with ChatClient

Call `stream()` instead of `call()` to get a reactive `Flux<String>` of response chunks.

java

```java
Flux<String> tokens = chatClient.prompt()
    .user(question)
    .stream()
    .content();
```

Streaming to the browser

Because the result is a `Flux`, expose it from a WebFlux endpoint with the server-sent-events media type and the tokens flow straight to the client as they arrive.

java

```java
@GetMapping(value = "/chat/stream",
            produces = MediaType.TEXT_EVENT_STREAM_VALUE)
Flux<String> stream(@RequestParam String q) {
    return chatClient.prompt().user(q).stream().content();
}
```

NOTE

Streaming pairs naturally with WebFlux (see ByteMart's Spring WebFlux guide). Even in an MVC app, Spring AI can stream, but a reactive stack makes end-to-end streaming cleanest.

When to stream

- Interactive chat UIs — perceived latency drops dramatically.
- Long generations — users see progress instead of a spinner.
- Not needed for short, backend-only extractions — use `call()` there.

Stream for responsiveness

stream() returns a Flux of tokens you can pipe to the browser over server-sent events, making chat UIs feel instant. Use call() for short, non-interactive backend tasks.

© 2026 ByteMart Digital

MODULE 3 · RAG & DATA

CHAPTER 10

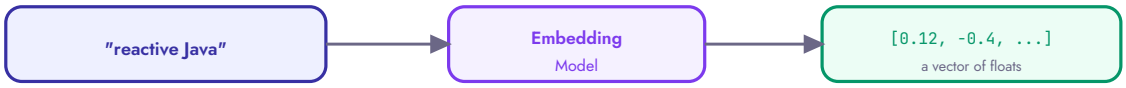
Embeddings

Turning text into vectors so machines can measure meaning and similarity.

Embeddings are the foundation of semantic search and RAG. An **embedding** converts a piece of text into a vector — a list of numbers — that captures its meaning.

Embedding
A numerical vector representation of text (or other data) such that semantically similar inputs map to nearby vectors in high-dimensional space.

Embeddings: Text as Vectors



An embedding turns text into a point in high-dimensional space.

Similar meanings sit close together — the basis of semantic search.

An embedding model maps text to a vector; similar meanings produce nearby vectors.

The EmbeddingModel

Spring AI's `EmbeddingModel` turns text into vectors. You rarely call it directly for RAG — the vector store does — but it is available when you need embeddings yourself.

```java
float[] vector = embeddingModel.embed("reactive programming in Java");
// e.g. a 1536-dimensional vector
```

Why vectors capture meaning

Because the model is trained so that related concepts sit close together, you can measure semantic similarity by the **distance** between vectors. 'Car' and 'automobile' land near each other; 'car' and 'banana' do not — even though the words share no letters.

Similarity

Similarity is usually measured by cosine distance between vectors. Searching becomes: embed the query, then find the stored vectors closest to it. This is **semantic** search — matching on meaning, not keywords.

Meaning as geometry
Embeddings turn text into vectors where distance reflects semantic similarity. They let you search by meaning rather than exact words — the building block of vector stores and RAG.

© 2026 ByteMart Digital

MODULE 3 · RAG & DATA

CHAPTER 11

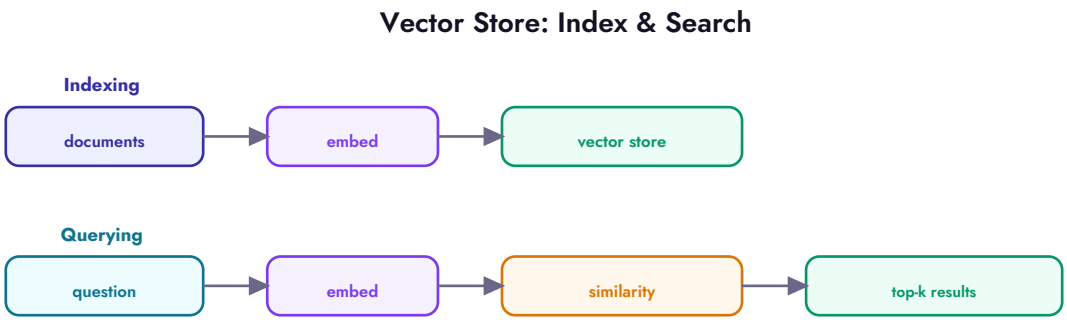
Vector Stores

Storing and searching embeddings to find the most relevant content by meaning.

Once text is embedded, you need somewhere to store the vectors and search them efficiently. That is a **vector store** — a database optimised for similarity search.

The VectorStore abstraction

Spring AI's `VectorStore` interface hides the specific database. Implementations include PgVector (PostgreSQL), Redis, Chroma, Milvus, Qdrant, and a `SimpleVectorStore` for in-memory development.



Documents are embedded and stored; a query is embedded and matched to the nearest vectors.

Adding documents

You add `Document` objects; the store embeds their text (via the configured `EmbeddingModel`) and persists the vectors along with metadata.

```java
vectorStore.add(List.of(
    new Document("Spring AI supports RAG.", Map.of("source", "docs")),
    new Document("WebFlux is non-blocking.", Map.of("source", "docs"))));
```

Similarity search

Query with natural language; the store embeds the query and returns the closest documents. You can cap the number of results and filter by metadata.

```java
List<Document> hits = vectorStore.similaritySearch(
    SearchRequest.builder()
        .query("How do I do retrieval?")
        .topK(4)
        .build());
```

TIP

For production, PgVector is a popular choice — it adds vector search to PostgreSQL, so you keep one database instead of running a separate vector service.

A database for meaning

A vector store persists embeddings and finds the nearest matches to a query. Spring AI's VectorStore abstraction lets you develop in-memory and deploy on PgVector, Redis, or others unchanged.

© 2026 ByteMart Digital

MODULE 3 · RAG & DATA

CHAPTER 12

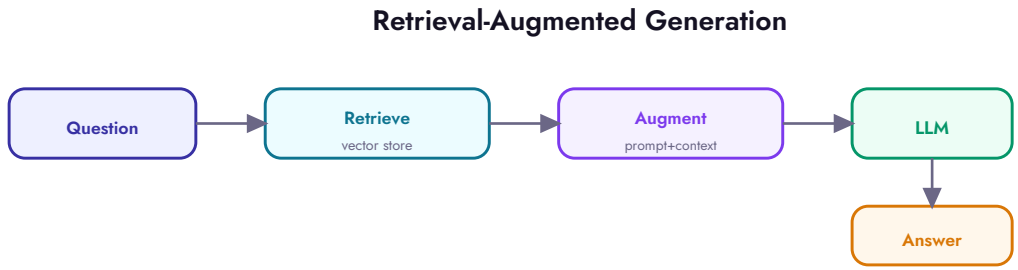
Retrieval-Augmented Generation (RAG)

Grounding the model in your own data to give accurate, sourced answers.

LLMs know only what they were trained on — not your documents, and not last week's data. **RAG** fixes this by retrieving relevant context from your data and feeding it to the model at query time.

> **Retrieval-Augmented Generation**
> A technique that retrieves relevant documents for a query and includes them in the prompt, so the model answers from your data rather than only its training — reducing hallucination and enabling citations.

The RAG flow



Ground the model in your own data: fetch relevant context, then answer from it.

Retrieve relevant context for the question, add it to the prompt, and let the model answer from it.

The steps are: embed the question, search the vector store for the most relevant chunks, insert those chunks into the prompt as context, and ask the model to answer using them.

RAG with the QuestionAnswerAdvisor

Spring AI packages this pattern as an **advisor**. Attach a `QuestionAnswerAdvisor` backed by your vector store and it automatically retrieves context and augments each prompt.

```java
String answer = chatClient.prompt()
    .advisors(new QuestionAnswerAdvisor(vectorStore))
    .user("What does our refund policy say?")
    .call()
    .content();
```

Why RAG matters

- **Accuracy** — answers grounded in real documents, not guesses.
- **Freshness** — update the store, not the model.
- **Citations** — you know which sources informed the answer.

- **Privacy** — your data stays in your store, sent only as needed.

Answer from your data

RAG retrieves relevant context and adds it to the prompt so the model answers from your documents. Spring AI's QuestionAnswerAdvisor makes it a one-line addition to a ChatClient call.

© 2026 ByteMart Digital

MODULE 3 · RAG & DATA

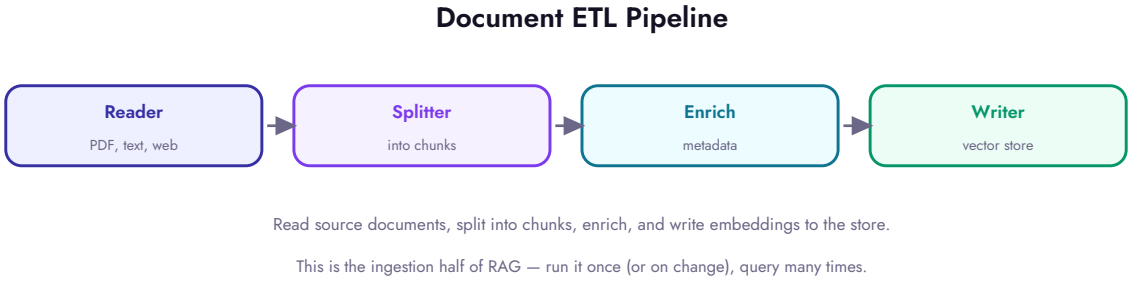
CHAPTER 13

The Document ETL Pipeline

Reading, splitting, enriching, and writing documents to a vector store for retrieval.

Before you can retrieve from your data, you must get it into the vector store. Spring AI provides an **ETL pipeline** — extract, transform, load — for turning raw documents into searchable embeddings.

The three stages



Read source documents, split them into chunks, enrich, and write embeddings to the vector store.

Stage	Component
Read	DocumentReader (PDF, text, web, JSON)
Transform	DocumentTransformer (splitter, enricher)
Write	DocumentWriter (the VectorStore)

Reading documents

Readers parse sources into `Document` objects. `TikaDocumentReader` handles many formats (PDF, Word, HTML); there are dedicated readers for PDFs, JSON, and plain text.

```java
var reader = new TikaDocumentReader(new ClassPathResource("policy.pdf"));
List<Document> docs = reader.get();
```

Splitting into chunks

Long documents must be split into smaller **chunks** so retrieval returns focused, relevant passages that fit the prompt. The `TokenTextSplitter` breaks documents on token boundaries.

```java
List<Document> chunks = new TokenTextSplitter().apply(docs);
vectorStore.add(chunks);   // embeds and stores
```

WATCH OUT

Chunk size matters. Too large and retrieval returns noisy, unfocused context; too small and you lose meaning. Tune it to your documents — a few hundred tokens is a common starting point.

Ingest before you retrieve

The ETL pipeline reads source documents, splits them into chunks, enriches them, and writes embeddings to the vector store. Chunking well is essential for good retrieval quality.

© 2026 ByteMart Digital

MODULE 3 · RAG & DATA

CHAPTER 14

Advanced RAG

Improving retrieval quality with query transformation and modular RAG advisors.

Basic RAG — embed the question, retrieve, augment — works, but real questions are messy. Advanced RAG improves each stage to retrieve better context and give better answers.

Why basic RAG falls short

- Vague or multi-part questions embed poorly and retrieve weak context.
- The best chunk may not literally resemble the question's wording.
- Irrelevant retrieved chunks can mislead the model.

Query transformation

Rewriting or expanding the query before retrieval improves results — rephrasing for clarity, expanding abbreviations, or splitting a compound question. Spring AI provides query transformers for this.

The RetrievalAugmentationAdvisor

Spring AI's modular RAG assembles a pipeline of components — query transformers, a document retriever, and context augmentation — into a single advisor you attach to the `ChatClient`.

```java
Advisor rag = RetrievalAugmentationAdvisor.builder()
    .documentRetriever(VectorStoreDocumentRetriever.builder()
        .vectorStore(vectorStore)
        .similarityThreshold(0.7)
        .topK(5)
        .build())
    .build();

chatClient.prompt().advisors(rag).user(question).call().content();
```

Tuning retrieval

Key levers are `topK` (how many chunks to retrieve), a **similarity threshold** (discard weak matches), and metadata filters (restrict to a source or tenant). Tuning these is where RAG quality is won.

> **Better retrieval, better answers**
> Advanced RAG transforms the query and tunes retrieval (topK, thresholds, filters) via the modular RetrievalAugmentationAdvisor. Retrieval quality, not the model, is usually the limiting factor.

© 2026 ByteMart Digital

MODULE 4 · TOOLS, MEMORY & AGENTS

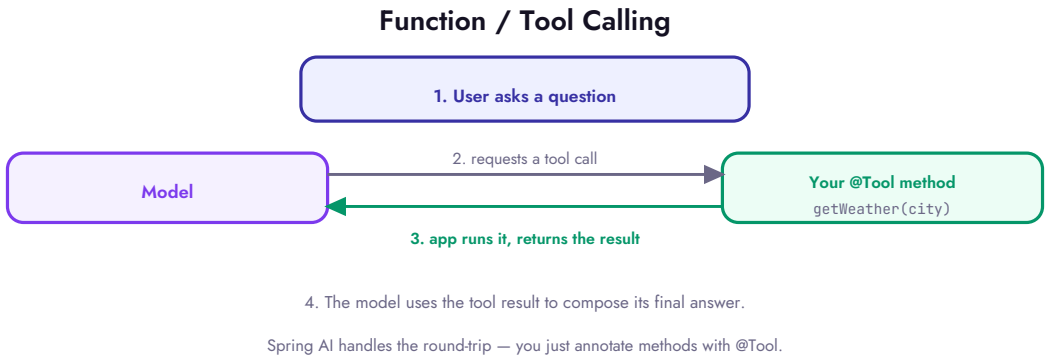
CHAPTER 15

Function / Tool Calling

Letting the model invoke your Java code to fetch data or take actions.

An LLM cannot check today's weather, query your database, or send an email — on its own. **Tool calling** lets the model ask your application to run a function and use the result, turning a chatbot into something that can act.

How it works



The model requests a tool, your app runs it, and the model uses the result to answer.

You expose methods as tools. When the model decides a tool is needed, it returns a request; Spring AI invokes your method, feeds the result back to the model, and the model composes its final answer — all handled for you.

Defining a tool

Annotate a method with `@Tool` and describe it. The description tells the model when and how to use it, so write it carefully.

```java
class WeatherTools {
    @Tool(description = "Get the current weather for a city")
    String getWeather(String city) {
        return weatherService.current(city);   // your real code
    }
}
```

Registering tools

Pass tool instances to the `ChatClient`. The model may call zero, one, or several during a single request.

java

```
chatClient.prompt()
    .user("What should I wear in Mumbai today?")
    .tools(new WeatherTools())
    .call().content();
```

WATCH OUT

Tools let the model trigger real actions. Treat tool inputs as untrusted, validate them, and be cautious granting tools that write data or spend money — the model decides when to call them.

Give the model hands

Tool calling exposes @Tool-annotated methods the model can invoke to fetch data or act. Spring AI manages the round-trip; you provide well-described, validated methods.

MODULE 4 · TOOLS, MEMORY & AGENTS

CHAPTER 16

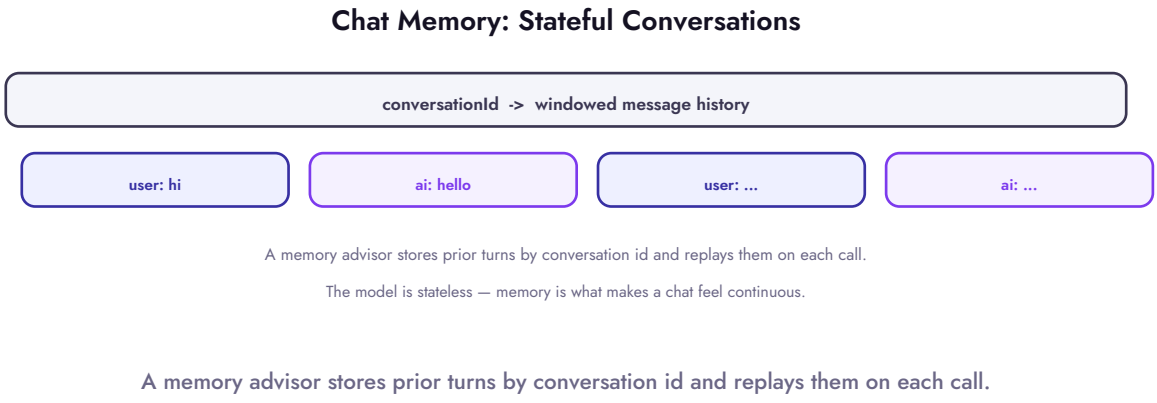
Chat Memory & Conversations

Making a stateless model feel like a continuous conversation.

A model has no memory — each API call is independent. To build a chat that remembers what was said, you must store the conversation and replay it. Spring AI's **chat memory** does this for you.

The problem

Ask 'What is Spring AI?' then 'Give me an example' and the second call has no idea what 'an example' refers to — unless you resend the first exchange. Memory automates that resending.



A memory advisor stores prior turns by conversation id and replays them on each call.

Memory advisors

Attach a chat-memory advisor (backed by a `ChatMemory`) and identify the conversation with an id. The advisor stores each turn and prepends recent history to every request.

```java
ChatMemory memory = MessageWindowChatMemory.builder().build();

chatClient.prompt()
    .advisors(MessageChatMemoryAdvisor.builder(memory).build())
    .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, userId))
    .user(message)
    .call().content();
```

Windowing and storage

- A **window** keeps only the most recent messages, controlling prompt size and cost.
- Memory can be stored in-memory, or persisted (JDBC, Cassandra) to survive restarts.
- Each conversation is isolated by its id, so users never see each other's history.

Replay history for continuity
Chat memory stores conversation turns by id and replays a recent window on each call, making a stateless model feel continuous. Choose in-memory or persistent storage and bound the window.

© 2026 ByteMart Digital

© 2026 ByteMart Digital

MODULE 4 · TOOLS, MEMORY & AGENTS

CHAPTER 17

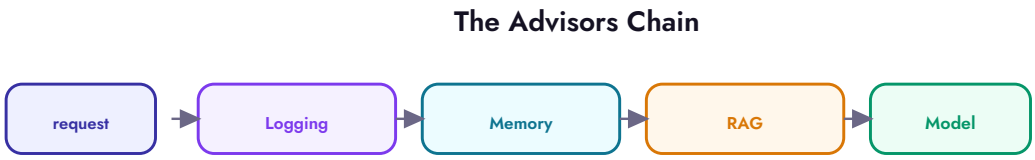
The Advisors API

The interceptor chain that powers RAG, memory, and logging in a composable way.

You have now used advisors for RAG and memory. The **Advisors API** is the general mechanism behind them — a chain of interceptors around every model call, much like servlet filters around a request.

What an advisor does

An advisor can inspect and modify the request before it reaches the model, and the response after — to inject context, record history, log, or transform output. Multiple advisors form an ordered chain.



Advisors intercept each request and response — a composable middleware chain around the model.

Built-in advisors

Advisor	Purpose
QuestionAnswerAdvisor	RAG retrieval
MessageChatMemoryAdvisor	conversation memory
SimpleLoggerAdvisor	log requests / responses
SafeGuardAdvisor	block disallowed content

Composing advisors

Register advisors on the builder (as defaults) or per request. They run in order, so you can layer logging around memory around RAG — each concern isolated and reusable.

```java
ChatClient client = builder
    .defaultAdvisors(
        new SimpleLoggerAdvisor(),
        MessageChatMemoryAdvisor.builder(memory).build(),
        new QuestionAnswerAdvisor(vectorStore))
    .build();
```

© 2026 ByteMart Digital

Writing your own

Implement the advisor interface to add custom cross-cutting behaviour — metrics, PII redaction, prompt rewriting — without touching your business logic.

Middleware for model calls

Advisors are a composable interceptor chain around every model call. RAG, memory, and logging are all advisors — layer the built-ins or write your own for custom concerns.

MODULE 4 · TOOLS, MEMORY & AGENTS

CHAPTER 18

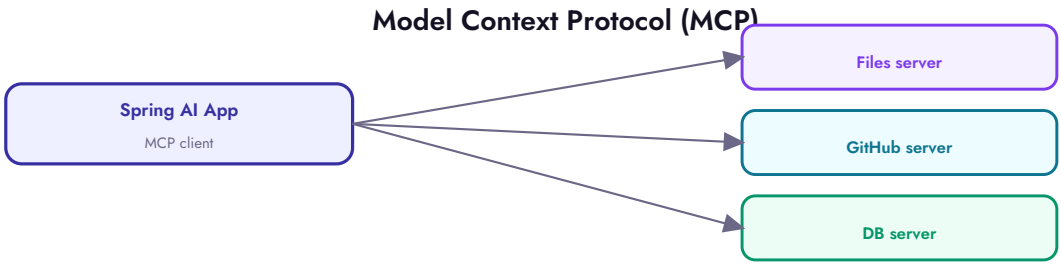
Model Context Protocol (MCP)

Connecting your application to external tools and data through an open standard.

The **Model Context Protocol** is an open standard for connecting AI applications to external tools and data sources. Spring AI provides both an MCP client and server, so your app can consume — or expose — capabilities over MCP.

> **`Model Context Protocol`**
> An open protocol that standardises how AI applications connect to external tools, resources, and data sources — letting a model use capabilities provided by separate MCP servers.

Client and server



MCP is a standard way to connect models to external tools and data over a common protocol.

Spring AI's MCP client exposes remote MCP tools to your model as callable tools.

Spring AI's MCP client connects your app to MCP servers that provide tools and data.

As an **MCP client**, your Spring AI app connects to MCP servers (for files, GitHub, databases, and more) and exposes their tools to the model as if they were local tools. As an **MCP server**, your app publishes its own tools for other AI applications to use.

Using MCP tools

Add the MCP client starter and configure the servers; Spring AI discovers their tools and makes them available to the model through the same tool-calling mechanism you already know.

```java
// tools discovered from configured MCP servers
chatClient.prompt()
    .user("Summarise the open issues in our repo.")
    .tools(mcpToolCallbacks)   // provided by the MCP client
    .call().content();
```

Why it matters

MCP turns integrations into reusable, standard building blocks. Instead of writing bespoke code for every data source, you connect to an MCP server — and the same server works with any MCP-compatible application.

A standard for AI integrations

MCP standardises how models reach external tools and data. Spring AI's MCP client exposes remote MCP tools to your model, and its server lets you publish your own — reusable across the ecosystem.

MODULE 4 · TOOLS, MEMORY & AGENTS

CHAPTER 19

Building Agents

Combining tools, memory, and reasoning into systems that accomplish multi-step goals.

An **agent** is an AI system that pursues a goal over multiple steps — reasoning about what to do, calling tools, observing results, and deciding the next action, rather than answering in one shot.

From chatbot to agent

A chatbot responds to a prompt. An agent loops: it plans, acts (via tools), observes the outcome, and repeats until the goal is met. Tool calling plus memory plus a well-designed prompt are the ingredients.

The building blocks in Spring AI

- **Tools** — the actions the agent can take (query, call an API, compute).
- **Memory** — so it remembers what it has already done.
- **Structured output** — to make decisions machine-readable.
- **Advisors** — to inject context and guardrails.

Effective patterns

Reliable agents favour explicit, constrained workflows over open-ended autonomy: chain focused steps, route requests to specialised handlers, and let the model orchestrate a small set of well-described tools. Simpler, bounded designs are more predictable and easier to debug.

WATCH OUT

Autonomous, open-ended agents are powerful but unpredictable and can loop, misfire tools, or run up cost. Start with constrained, well-scoped workflows and add autonomy only where it clearly pays off.

Guardrails

Bound the number of steps, validate every tool input, cap spending, and log each decision. An agent that can act must be observable and containable — treat its tool calls with the same rigour as any code that touches production.

Goals over multiple steps

Agents combine tools, memory, and reasoning to accomplish multi-step goals. Build them from Spring AI's tool calling and advisors — and favour bounded, well-scoped workflows with strong guardrails.

© 2026 ByteMart Digital

MODULE 5 · MULTIMODAL & PROVIDERS

CHAPTER 20

Multimodality & Image Models

Working with images as input and generating images as output.

Modern models are **multimodal** — they handle more than text. Spring AI lets you send images to a model for analysis and generate images from a text prompt.

Images as input

Attach media to a user message and a vision-capable model can describe, analyse, or answer questions about the image — useful for OCR, content moderation, or visual Q&A.;

```java
chatClient.prompt()
    .user(u -> u.text("What is in this image?")
                .media(MimeTypeUtils.IMAGE_PNG, imageResource))
    .call().content();
```

Generating images

The `ImageModel` abstraction generates images from a text prompt, returning URLs or base64 data you can store or display.

```java
ImageResponse resp = imageModel.call(
    new ImagePrompt("a minimalist logo of a blue byte"));
String url = resp.getResult().getOutput().getUrl();
```

Use cases for a backend

- Extract text and data from scanned documents and receipts.
- Moderate or tag user-uploaded images.
- Generate thumbnails, illustrations, or marketing visuals.
- Answer questions about diagrams or screenshots.

Beyond text
Multimodal models accept images as input (analysis, OCR, visual Q&A;) and the ImageModel generates images from prompts — both behind Spring AI's consistent abstractions.

MODULE 5 · MULTIMODAL & PROVIDERS

CHAPTER 21

Audio: Transcription & Speech

Converting speech to text and text to speech within a Spring application.

Spring AI also abstracts audio models, letting you add voice features — transcribing recordings to text and synthesising text into spoken audio.

Speech to text (transcription)

A transcription model converts an audio file into text — the basis for voice notes, meeting summaries, and voice commands.

```java
String text = transcriptionModel.call(
    new AudioTranscriptionPrompt(audioResource))
    .getResult().getOutput();
```

Text to speech

A speech model turns text into audio you can stream or store — for accessibility, voice assistants, or narrated content.

```java
byte[] audio = speechModel.call(
    new SpeechPrompt("Welcome to ByteMart."))
    .getResult().getOutput();
```

A voice pipeline

Combine the pieces for a full voice assistant: transcribe the user's speech, feed the text to a `ChatClient` (with RAG and tools), then synthesise the reply back to audio — all with Spring AI abstractions.

> **Give your app a voice**
> Spring AI's transcription and speech models convert audio to text and back. Chain them with a ChatClient to build a complete voice pipeline inside a Spring Boot service.

MODULE 5 · MULTIMODAL & PROVIDERS

CHAPTER 22

Working with Local Models

Running open models on your own hardware for privacy, cost control, and offline use.

Not every use case should send data to a hosted API. Running **local models** — via Ollama and open-weight models — keeps data on your infrastructure and removes per-token costs.

Why run locally

- **Privacy** — sensitive data never leaves your network.
- **Cost** — no per-call charges after the hardware.
- **Offline** — works without internet access.
- **Control** — pin a specific model version, no surprise changes.

Ollama with Spring AI

Ollama runs open models (Llama, Mistral, and others) locally and exposes an API. Spring AI's Ollama starter targets it — the same `ChatClient` code, pointed at your machine.

properties

```
spring.ai.ollama.base-url=http://localhost:11434
spring.ai.ollama.chat.options.model=llama3.1
```

The trade-offs

Local open models are improving fast but generally trail the best hosted models in raw capability, and they need capable hardware (GPU/RAM) for good speed. A common strategy is hybrid: local models for routine or sensitive tasks, hosted models for the hardest ones.

TIP

Because Spring AI is provider-portable, develop and test against a free local Ollama model, then switch to a hosted provider for production by changing configuration — no code changes.

Keep it in-house when it matters

Local models via Ollama give privacy, cost control, and offline operation at some cost in capability and hardware. Spring AI's portability makes mixing local and hosted models trivial.

MODULE 6 · PRODUCTION

CHAPTER 23

Observability, Evaluation & Testing

Measuring, validating, and testing AI features that are inherently non-deterministic.

AI features are non-deterministic, so the usual assertions do not fit neatly. You need observability to see what the model does, evaluation to judge quality, and testing strategies suited to variable output.

Observability

Spring AI integrates with Micrometer to record observations for model calls — latency, token usage, and errors — surfaced through Actuator and your metrics backend. Tracking token usage is essential, since it drives cost.

- Latency and error rate per model call.
- Prompt and completion token counts (and therefore cost).
- Which advisors and tools ran for a request.

Evaluation

Because you cannot assert exact output, evaluate **quality** instead. Spring AI provides evaluators — for example a relevancy evaluator that checks whether an answer is grounded in the retrieved context — often using a model to judge a model.

```java
RelevancyEvaluator evaluator = new RelevancyEvaluator(builder);
EvaluationResponse result = evaluator.evaluate(
    new EvaluationRequest(userText, context, answer));
assertTrue(result.isPass());
```

Testing strategies

- Assert on **structure and constraints**, not exact wording.
- Use structured output so you can validate typed fields.
- Evaluate a set of representative cases and track a pass rate over time.
- Mock the model in unit tests; use real models in a small evaluation suite.

> **Measure quality, not exact text**
> Instrument model calls for latency, tokens, and cost; evaluate answer quality with evaluators; and test on structure and pass rates rather than exact output. Non-determinism changes how you verify.

© 2026 ByteMart Digital

MODULE 6 · PRODUCTION

CHAPTER 24

Production Concerns & Best Practices

Cost, safety, reliability, and the practices that make AI features production-ready.

Shipping an AI feature means handling concerns a normal API does not have: unpredictable cost, prompt injection, hallucination, and rate limits. A few disciplined practices keep it safe and affordable.

Managing cost

- Track token usage per feature; it is the main cost driver.
- Trim prompts and cap chat-memory windows — shorter prompts cost less.
- Cache responses for repeated queries where appropriate.
- Route easy tasks to cheaper (or local) models.

Safety and security

Treat model output as untrusted and user input as potentially adversarial. **Prompt injection** — where user text tries to override your instructions — is a real threat, especially with tools and RAG.

WATCH OUT
Never let a model's output trigger a dangerous action unchecked. Validate tool inputs, constrain what tools can do, sanitise retrieved content, and keep humans in the loop for high-impact actions.

Reliability

- Apply timeouts, retries, and fallbacks to model calls — they are remote and can fail.
- Handle rate limits with backoff.
- Ground answers with RAG and validate structured output to reduce hallucination.
- Provide a graceful degraded response when the model is unavailable.

Best practices recap

- Externalise prompts and iterate on them like code.
- Ground the model in your data; do not rely on its training for facts.
- Instrument everything — tokens, latency, quality.
- Stay provider-portable so you can adapt to a fast-moving field.

Ship AI responsibly
Production AI demands cost control, defence against prompt injection, resilient model calls, and grounding to curb hallucination. Instrument, validate, and keep humans in the loop for high-impact actions.

© 2026 ByteMart Digital